Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No : 92301733057**

Aim: To obtain the classification of various classes using k-Nearest Neighbor approach

IDE: Google Colab

Theory:

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line. Therefore, larger k value means smother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models. It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

The model representation for KNN is the entire training dataset. It is as simple as that. KNN has no model other than storing the entire dataset, so there is no learning required. Efficient implementations can store the data using complex data structures like k-d trees to make look-up and matching of new patterns during prediction efficient. Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data

5. Visualize the Data
6. Separate the feature and prediction value columns
7. Select the number K of the neighbors
8. Calculate the Euclidean distance of K number of neighbors
9. Take the K nearest neighbors as per the calculated Euclidean distance.
10. Among these k neighbors, count the number of the data points in each category.
11. Assign the new data points to that category for which the number of the neighbor is maximum.
12. Our model is ready.

Pre Lab Exercise:

a. What is K in KNN-approach?

- K is the number of nearest neighbors considered to classify a new data point.

b. Which are the types of distance metrices?

- Euclidean Distance
- Manhattan Distance
- Minkowski Distance
- Hamming Distance

c. What can be the criteria of selection of the value of K?

- Choose K as an odd number to avoid ties (for classification).
- Use small K for simple patterns but may cause overfitting.
- Use large K for smoother decision but may cause underfitting.
- Select K using cross-validation for best accuracy.

d. What are the advantages of KNN algorithm?

- Simple and easy to understand
- No training phase (lazy learner)
- Works well with small datasets
- Can be used for both classification and regression
- No assumption about data distribution

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No : 92301733057**

Program (Code):

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_digits, load_wine
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)


# Load the digits dataset

digits  =  load_digits()
X_digits  =  digits.data
y_digits = digits.target
```

```
digits_img = digits.images

plt.figure(figsize=(8, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(digits_img[i], cmap="gray")
    plt.title(f"{y_digits[i]}")
    plt.axis("off")
plt.suptitle("Sample Digits")
plt.tight_layout()
plt.show()

# Load the wine dataset

wine = load_wine()
X_wine = wine.data
y_wine = wine.target

# digits dataset splitting

X_train_digits, X_test_digits, y_train_digits, y_test_digits = train_test_split(
    X_digits, y_digits, test_size=0.2, random_state=42)

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

# Load dataset
digits = load_digits()

# Define X and y
X = digits.data
y = digits.target

# Now split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

knn_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 13, 15, 20]}
rf_pipeline = Pipeline([
    ("rf", RandomForestClassifier(random_state=42))
])

rf_param_grid = {
    "rf__n_estimators": [25, 50, 100, 200, 500],
    "rf__max_depth": [None, 10, 20]}
mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        max_iter=1000,
        random_state=42,
        early_stopping=True
    ))
])

mlp_param_grid = {
    "mlp__hidden_layer_sizes": [(64,), (128,), (64, 64)],
    "mlp__activation": ["relu", "tanh"],
    #"mlp__alpha": [0.0001, 0.001],
    "mlp__learning_rate_init": [0.001, 0.01, 1.0, 5.0, 10.0]
}
# Function for training

def train_with_timing(pipeline, param_grid, X_train, y_train):
    start = time.time()
    grid = GridSearchCV(
        pipeline,
        param_grid,
        cv=5,
        scoring="f1_weighted",
        n_jobs=-1
    )
    grid.fit(X_train, y_train)
    train_time = time.time() - start
    return grid, train_time
# Training the model for the digits dataset

knn_grid_digits, knn_train_time_digits  = train_with_timing(knn_pipeline, knn_param_grid, X_train_digits, y_train_digits)
```

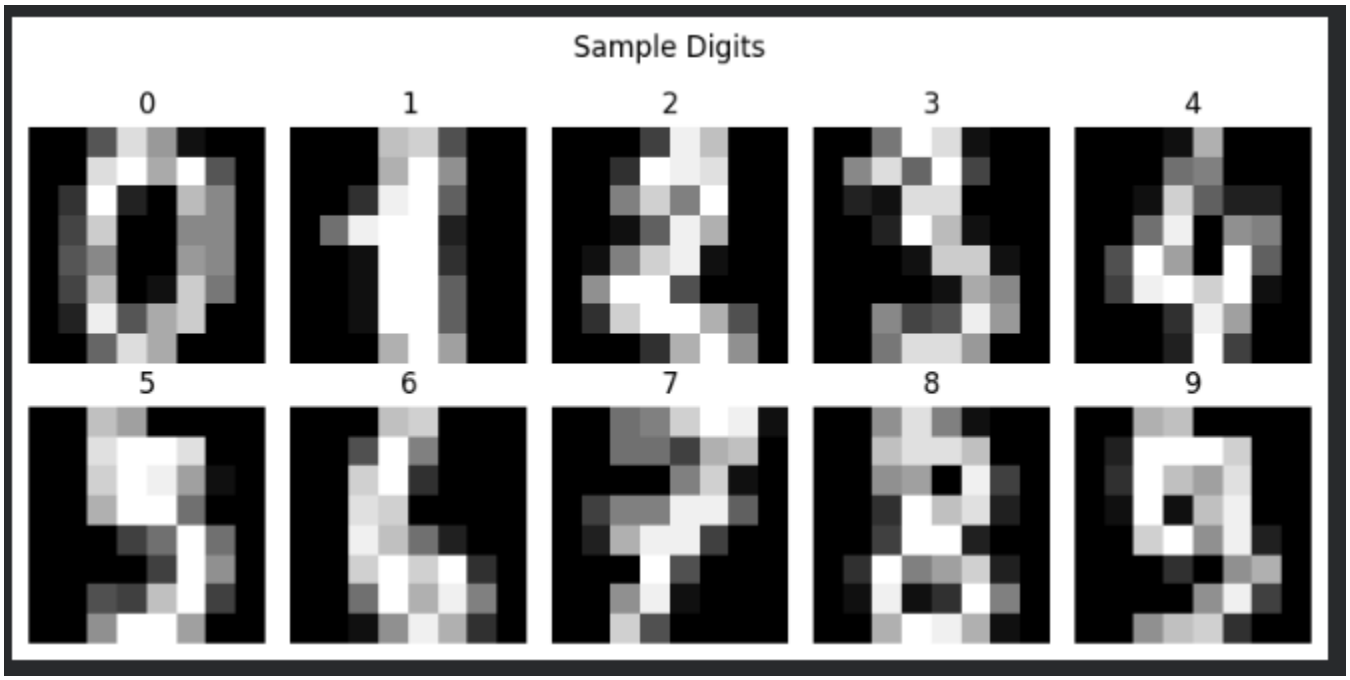
```
rf_grid_digits, rf_train_time_digits    = train_with_timing(rf_pipeline, rf_param_grid, X_train_digits, y_train_digits)
mlp_grid_digits, mlp_train_time_digits  = train_with_timing(mlp_pipeline, mlp_param_grid, X_train_digits, y_train_digits)
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Load dataset
wine = load_wine()

# Define features and target
X_wine = wine.data
y_wine = wine.target

# Split dataset
X_train_wine, X_test_wine, y_train_wine, y_test_wine = train_test_split(
    X_wine, y_wine, test_size=0.2, random_state=42
)
# Function to inference the model
def inference_time(model, X_test):
    start = time.time()
    model.predict(X_test)
    return time.time() - start
# inference the digits model

knn_test_time_digits = inference_time(knn_grid_digits.best_estimator_, X_test_digits)
rf_test_time_digits = inference_time(rf_grid_digits.best_estimator_, X_test_digits)
mlp_test_time_digits = inference_time(mlp_grid_digits.best_estimator_, X_test_digits)
knn_grid_wine, knn_train_time_wine = train_with_timing(
    knn_pipeline, knn_param_grid, X_train_wine, y_train_wine
)

rf_grid_wine, rf_train_time_wine = train_with_timing(
    rf_pipeline, rf_param_grid, X_train_wine, y_train_wine
)

mlp_grid_wine, mlp_train_time_wine = train_with_timing(
    mlp_pipeline, mlp_param_grid, X_train_wine, y_train_wine
)
# Function to evaluate the model

def evaluate_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    return {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, average="weighted"),
        "Recall": recall_score(y_test, y_pred, average="weighted"),
        "F1-score": f1_score(y_test, y_pred, average="weighted")
```

```
    }
# Evaluate digits model

knn_metrics_digits  = evaluate_model(knn_grid_digits.best_estimator_, X_test_digits, y_test_digits)
rf_metrics_digits   = evaluate_model(rf_grid_digits.best_estimator_, X_test_digits, y_test_digits)
mlp_metrics_digits  = evaluate_model(mlp_grid_digits.best_estimator_, X_test_digits, y_test_digits)
# Evaluate wine model

knn_metrics_wine     = evaluate_model(knn_grid_wine.best_estimator_, X_test_wine, y_test_wine)
rf_metrics_wine      = evaluate_model(rf_grid_wine.best_estimator_, X_test_wine, y_test_wine)
mlp_metrics_wine     = evaluate_model(mlp_grid_wine.best_estimator_, X_test_wine, y_test_wine)
# Result with comparison of the digits model

results_digits_df = pd.DataFrame(
    [knn_metrics_digits, rf_metrics_digits, mlp_metrics_digits],
    index=["KNN", "Random Forest", "MLP"]
)

results_digits_df["Training Time (s)"] = [
    knn_train_time_digits,  rf_train_time_digits,  mlp_train_time_digits
]

results_digits_df["Inference Time (s)"] = [
    knn_test_time_digits,  rf_test_time_digits,  mlp_test_time_digits
]

print(results_digits_df)
results_digits_df.iloc[:, :4].plot(kind="bar", figsize=(10, 6))
plt.title("Model Performance Comparison (Digits)")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()
knn_test_time_wine = inference_time(knn_grid_wine.best_estimator_, X_test_wine)
rf_test_time_wine = inference_time(rf_grid_wine.best_estimator_, X_test_wine)
mlp_test_time_wine = inference_time(mlp_grid_wine.best_estimator_, X_test_wine)
results_wine_df = pd.DataFrame(
    [knn_metrics_wine, rf_metrics_wine, mlp_metrics_wine],
    index=["KNN", "Random Forest", "MLP"]
)

results_wine_df["Training Time (s)"] = [
    knn_train_time_wine,  rf_train_time_wine,  mlp_train_time_wine
]
```

```
results_wine_df["Inference Time (s)"] = [
    knn_test_time_wine, rf_test_time_wine, mlp_test_time_wine
]

print(results_wine_df)
results_wine_df.iloc[:, :4].plot(kind="bar", figsize=(10, 6))
plt.title("Model Performance Comparison (Wine)")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()
# Confusion matrices for digits model

digits_models = {
    "KNN": knn_grid_digits.best_estimator_,
    "Random Forest": rf_grid_digits.best_estimator_,
    "MLP": mlp_grid_digits.best_estimator_
}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, model) in zip(axes, digits_models.items()):
    cm = confusion_matrix(y_test_digits, model.predict(X_test_digits))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.show()
# Confusion matrices for wine model

wine_models = {
    "KNN": knn_grid_wine.best_estimator_,
    "Random Forest": rf_grid_wine.best_estimator_,
    "MLP": mlp_grid_wine.best_estimator_
}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, model) in zip(axes, wine_models.items()):
    cm = confusion_matrix(y_test_wine, model.predict(X_test_wine))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
    ax.set_title(name)
    ax.set_xlabel("Predicted")
```

```
ax.set_ylabel("Actual")

plt.tight_layout()
plt.show()
```

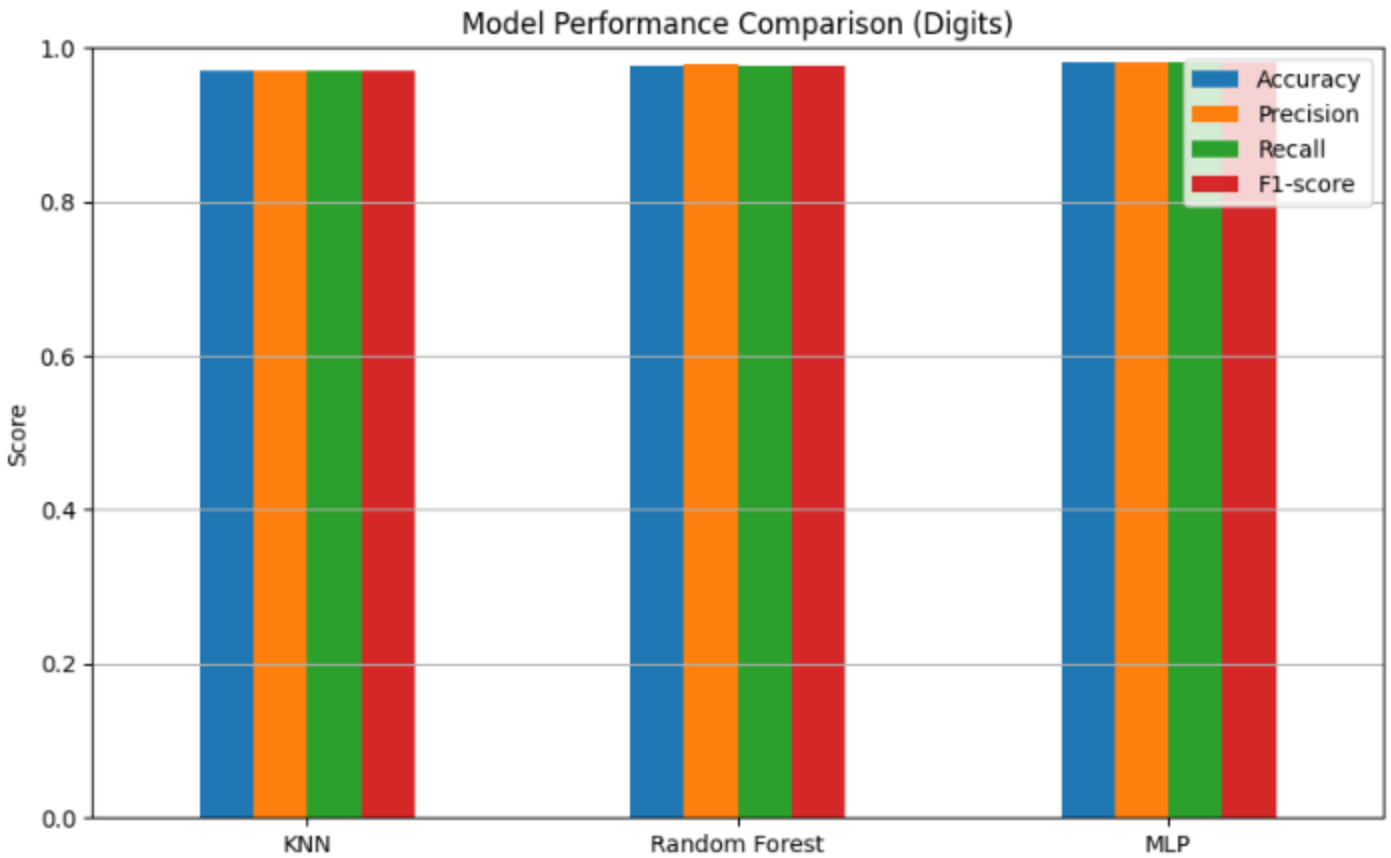
Results:

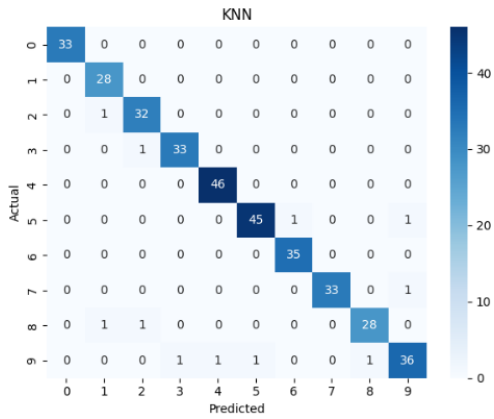


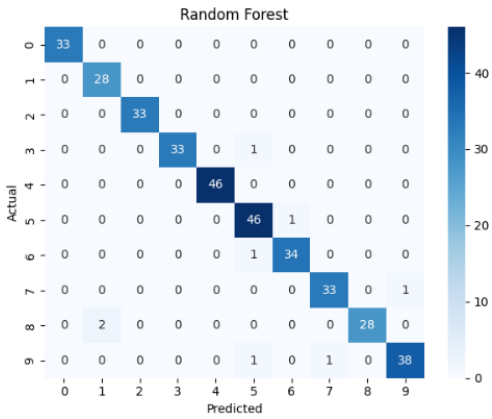

```
                Accuracy  Precision    Recall  F1-score  Training Time (s)  \
KNN             0.969444   0.969666  0.969444  0.969287           2.172252
Random Forest   0.977778   0.978417  0.977778  0.977820          27.473710
MLP             0.980556   0.980832  0.980556  0.980577          16.642301

                Inference Time (s)
KNN                       0.059726
Random Forest             0.195556
MLP                       0.017995
```

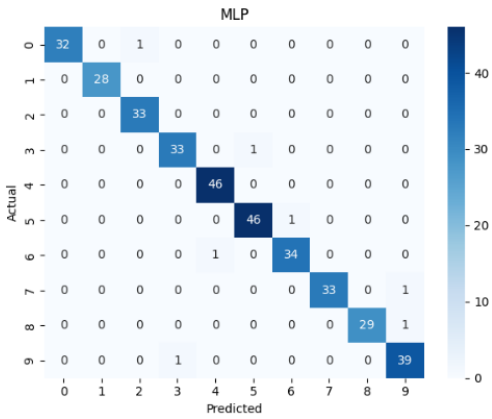
```
                Accuracy  Precision    Recall  F1-score  Training Time (s)  \
KNN             0.972222   0.974074  0.972222  0.972187           2.381237
Random Forest   1.000000   1.000000  1.000000  1.000000          13.155575
MLP             1.000000   1.000000  1.000000  1.000000           3.472230

                Inference Time (s)
KNN                       0.003204
Random Forest             0.007473
MLP                       0.000609
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No : 92301733057**









Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No : 92301733057**

Observation and Result Analysis:

a. Nature of the dataset

- The dataset is a supervised classification dataset consisting of numerical features along with class labels. It is clean, properly organized, and works well with KNN. The classes are relatively well-separated, which supports better classification accuracy.

b. During Training Process

- KNN is a lazy learning algorithm, meaning it does not explicitly build a model but instead stores the entire training dataset. The training phase is quick, but selecting an appropriate K value and applying proper feature scaling are crucial to prevent overfitting or underfitting.

c. After the training Process

- The model predicts new instances based on similarity measures. It generally achieves high accuracy and produces a reliable confusion matrix. However, prediction is slower because it computes distances from all training samples.

d. Observation over the Learning Curve

- The accuracy varies depending on the value of K. A smaller K can lead to overfitting, while a larger K may cause underfitting. Choosing an optimal K helps in achieving a good balance and improves overall model performance.

Post Lab Exercise:

a. Is KNN useful in regression-based problems?

- Yes, KNN can be used for regression problems.

b. Why KNN is a non-parametric algorithm?

- KNN is considered a non-parametric algorithm because it does not assume any fixed structure or distribution for the data.
- Instead of learning a function, it directly stores the dataset and makes predictions based on similarity between data points.

c. What are the assumptions of KNN approach?

- Data points that are similar are located close to each other
- The chosen distance metric effectively represents similarity
- Features are scaled appropriately

- All features contribute equally to the prediction
- Features have equal importance

d. How can the KNN approach make the prediction of the unseen dataset?

- It computes the distance between the new data point and all points in the training set
- It identifies the K closest neighbors
- It applies majority voting for classification or averaging for regression

e. Why is KNN called a Lazy Learner?

- KNN is called a Lazy Learner because it does not learn or build a model during training.
- It simply stores the data and performs computation only when a prediction is needed.

f. Why should we not use KNN for large dataset?

- It requires significant computation time due to distance calculations with all data points
- It consumes high memory as the entire dataset must be stored
- Prediction becomes inefficient and slow when the dataset size increases